

# Guide 0 — Les bases de Laravel dans notre projet

Pour les 4 membres. À lire en premier (environ 45 min). Tous les exemples viennent de notre repo.

## 1. Laravel expliqué à un développeur Spring Boot

Vous connaissez déjà les concepts. Seuls les noms changent :

| Spring Boot / Angular                   | Laravel (notre projet)                                                | Où dans le repo                           |
|-----------------------------------------|-----------------------------------------------------------------------|-------------------------------------------|
| @Entity + JPA                           | <b>Modèle Eloquent</b>                                                | app/Models/Signalement.php                |
| JpaRepository                           | Le modèle lui-même :<br>Signalement::where(...)->get()                | —                                         |
| Flyway / Liquibase                      | <b>Migrations</b>                                                     | database/migrations/                      |
| @RestController +<br>template Thymeleaf | <b>Composant Livewire</b> (logique PHP<br>+ vue dans un seul fichier) | resources/views/pages/...                 |
| Composant Angular                       | Composant Livewire (même idée :<br>état + template + événements)      | idem                                      |
| @PreAuthorize / Spring<br>Security      | <b>Policy</b> + \$this->authorize()                                   | app/Policies/                             |
| Bean Validation @NotNull                | Règles de validation rules()                                          | dans chaque composant                     |
| application.properties                  | <b>.env</b> + config/                                                 | racine du projet                          |
| Données de test                         | <b>Factory + Seeder</b>                                               | database/factories/ ,<br>database/seeder/ |
| JUnit                                   | <b>Pest</b>                                                           | tests/Feature/                            |
| Back-office maison                      | <b>Filament</b> (généré)                                              | app/Filament/ → /admin                    |

## 2. Le trajet d'une requête

Quand un utilisateur ouvre /signalements :

- Route** ( routes/features.php ) : l'URL pointe vers le composant pages::signalements.index . Le groupe auth exige d'être connecté, sinon redirection vers /login .
- Composant Livewire** ( resources/views/pages/signalements/index.blade.php ) : mount() s'exécute. On y vérifie les droits avec \$this->authorize(...) .
- Policy** ( app/Policies/SignalementPolicy.php ) : répond oui ou non. Non → erreur 403.
- Modèle** : on lit les données ( Signalement::query()->latest()->paginate(10) ).
- Vue Blade** (bas du même fichier) : le HTML est rendu avec les composants **Flux** ( <flux:table> , <flux:button> ...).

6. Ensuite, chaque clic ( `wire:click` ) ou saisie ( `wire:model` ) renvoie une petite requête AJAX au composant, **sans recharger la page**. Livewire met à jour le HTML tout seul.

### 3. Les dossiers à connaître (et rien d'autre)

```
app/Models/           ← entités (tables)
app/Policies/         ← qui a le droit de faire quoi
app/Filament/         ← espace admin généré
app/Console/Commands/ ← nos commandes (make:feature)
database/migrations/   ← structure des tables
database/factories/    ← fausses données réalistes
database/seeders/      ← remplissage de la base de démo
resources/views/pages/ ← toutes les pages (composants Livewire)
resources/views/layouts/app/sidebar.blade.php ← menu de gauche
routes/features.php    ← routes de nos fonctionnalités
tests/Feature/         ← tests Pest
CLAUDE.md, CONVENTION.md ← règles de l'équipe
```

### 4. Un modèle Eloquent

```
#[Fillable(['titre', 'description', 'niveau', 'photo'])]
class Signalement extends Model
{
    use HasFactory;

    public const NIVEAU_OPTIONS = ['faible', 'moyen', 'critique'];

    public function user(): BelongsTo // un signalement appartient à un utilisateur
    {
        return $this->belongsTo(User::class);
    }

    protected function casts(): array // conversions automatiques
    {
        return ['date_incident' => 'date'];
    }
}
```

- `#[Fillable]` = les champs qu'on a le droit de remplir depuis un formulaire. `user_id` et `role` n'y sont **jamais** : on les assigne dans le code. C'est une protection de sécurité, pas un oubli.
- Requêtes courantes :

```

Signalement::all(); // tout
Signalement::find(5); // par id (ou null)
Signalement::findOrFail(5); // par id (ou 404)
Signalement::where('niveau', 'critique')->latest()->get();
Signalement::whereBelongsTo(auth()->user())->count(); // ceux de l'utilisateur connecté
$signalement->user->name; // relation
$signalement->update(['titre' => 'Nouveau']);
$signalement->delete();

```

## 5. Une migration

```

Schema::create('signalements', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->string('titre');
    $table->text('description')->nullable();
    $table->string('niveau', 50)->index();
    $table->timestamps(); // created_at, updated_at
});

```

- `php artisan migrate` applique les nouvelles migrations.
- **Règle d'or : ne jamais modifier une migration déjà déployée sur le serveur.** Pour ajouter une colonne, on crée une nouvelle migration : `php artisan make:migration add_statut_to_signalements_table`.

## 6. Un composant Livewire (le cœur de notre front)

Un fichier = la logique en haut, la vue en bas :

```

<?php
use App\Models\Signalement;
use Livewire\Attributes\Title;
use Livewire\Component;

new #[Title('Signalements')] class extends Component {
    public string $search = ''; // état, lié à un champ

    public function mount(): void // au chargement
    {
        $this->authorize('viewAny', Signalement::class);
    }

    public function delete(int $id): void // appelé par wire:click
    {
        $signalement = Signalement::findOrFail($id);
        $this->authorize('delete', $signalement);
        $signalement->delete();
    }
}; ?>

<section>
    <flux:input wire:model.live="search" placeholder="Rechercher..." />
    <flux:button wire:click="delete({{ $item->id }})">Supprimer</flux:button>
</section>

```

- `wire:model="x"` relie un champ à la propriété `$x`. `.live` = à chaque frappe ; `.live.debounce.300ms` = après une pause.
- `wire:click="methode"` appelle une méthode PHP.
- `wire:submit="save"` sur un `<form>` appelle `save()`.
- **Toute méthode publique peut être appelée depuis le navigateur**, avec n'importe quel argument. C'est pour ça que chaque méthode vérifie les droits.

## 7. Blade en 30 secondes

```

{{ $signalement->titre }}           {{-- affiche, en échappant le HTML (sûr) --}}
@if ($liste->isEmpty()) ... @endif
@foreach ($liste as $item) ... @endforeach
@can('update', $signalement) ... @endcan  {{-- n'affiche que si la Policy l'autorise --}}
{{ route('signalements.show', $signalement) }}  {{-- URL d'une route nommée --}}

```

Jamais `{!! $variable !!` avec une donnée saisie par un utilisateur (faille XSS).

## 8. Policy et validation

```
// app/Policies/SignalementPolicy.php
public function update(User $user, Signalement $signalement): bool
{
    return $user->isAdmin() || $signalement->user_id === $user->id;
}

// dans le composant
protected function rules(): array
{
    return [
        'titre' => ['required', 'string', 'max:255'],
        'niveau' => ['required', Rule::in(Signalement::NIVEAU_OPTIONS)],
        'photo' => ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],
    ];
}

$validated = $this->validate(); // renvoie les données propres, ou affiche les erreurs
```

## 9. Notre arme : le générateur

```
php artisan make:feature Signalement --
fields="titre:string,description:text?,niveau:enum(faible/moyen/critique),photo:image?,latitude
:decimal?,longitude:decimal?" --icon=exclamation-triangle
php artisan migrate
```

Il crée en une fois : migration, modèle, factory, policy, 3 pages (liste, formulaire, détail), routes, entrée de menu, données de démo et 6 tests. **On génère, puis on adapte.**

Types : `string`, `text`, `integer`, `decimal`, `boolean`, `date`, `datetime`, `enum(a/b/c)`, `image`. Suffixe `?` = facultatif.

## 10. Travailler avec Claude Code

Le fichier `CLAUDE.md` donne à Claude Code tout le contexte du projet et de la compétition. Commandes préparées (à copier une fois dans `.claude/commands/`) :

| Commande                                         | Usage                                                        |
|--------------------------------------------------|--------------------------------------------------------------|
| <code>/triage &lt;annonce&gt;</code>             | Analyse une fonctionnalité annoncée : priorité, effort, plan |
| <code>/fonctionnalite &lt;description&gt;</code> | Implémente en respectant nos règles, tests compris           |
| <code>/audit-secu [zone]</code>                  | Audit de sécurité avant livraison                            |
| <code>/recap-jury &lt;liste&gt;</code>           | Récap des fonctionnalités + script vidéo                     |

Règles : **relisez ce que Claude Code produit**, lancez vous-même les tests, et c'est vous qui committez.

## 11. Commandes du quotidien

```
git pull # récupérer le travail des autres
composer install ; npm install # si les dépendances ont changé
php artisan migrate # si de nouvelles migrations
composer run dev # lancer le projet (laisser tourner)
php artisan migrate:fresh --seed # repartir d'une base propre (LOCAL uniquement)
vendor/bin/pint # formatage – avant chaque push
php artisan test # tests – avant chaque push
php artisan route:list --path=signalements # voir les routes
php artisan tinker # console PHP sur l'application
```

## 12. Déboguer vite

| Symptôme                         | Réflexe                                                                   |
|----------------------------------|---------------------------------------------------------------------------|
| Page blanche / erreur 500        | Lire la fin de <code>storage/logs/laravel.log</code>                      |
| Changement de style invisible    | <code>composer run dev</code> doit tourner, ou <code>npm run build</code> |
| « Route not defined »            | <code>php artisan route:list</code> ; vérifier le nom de la route         |
| Champ non enregistré             | Est-il dans <code>#[Fillable]</code> ? dans <code>rules()</code> ?        |
| 403 inattendu                    | Relire la Policy et l'appel <code>authorize</code>                        |
| « Column not found »             | Migration non lancée : <code>php artisan migrate</code>                   |
| Modification non prise en compte | <code>php artisan optimize:clear</code>                                   |

`dd($variable);` affiche une variable et arrête l'exécution. À retirer avant de commiter.